ML-Project Report

Master of Computer Application

Semester – II

Topic:

CYBERBULLYING DETECTION

USING ENSEMBLE LEARNING

Submitted By :

SANJANA R
251102250055

Department of Computer ApplicationAlliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106

GIT HUB REPO LINK= https://github.com/saraswati-sanj/ML_FINAL_PROJECT/tree/main

1. Abstract

This project presents a machine learning-based system for the automatic detection and classification of cyberbullying content on social media platforms, specifically Twitter. Using a dataset of 47,692 tweets, the system classifies text into six categories: age-based, ethnicity-based, gender-based, religion-based, other cyberbullying, and non-cyberbullying content.

The approach combines TF-IDF vectorization with an ensemble of machine learning models — Logistic Regression and Linear SVC — evaluated through Hard Voting, Soft Voting, and Stacking. Soft Voting achieved the best overall accuracy of 83.83%, while individual classifiers and stacking also delivered competitive results above 83%.

2. Introduction

* Background

Cyberbullying — the use of digital communication channels to harass, threaten, or intimidate individuals — has become a significant social problem with the widespread adoption of social media. Unlike traditional bullying, cyberbullying occurs 24/7, can reach large audiences instantly, and often involves anonymity, making it particularly harmful.

Manually moderating millions of social media posts is infeasible. Automated detection using Natural Language Processing (NLP) and machine learning offers a scalable solution to identify and flag harmful content in real time.

*Objectives

- Build an automated classifier to detect cyberbullying in tweets.
- Categorize cyberbullying into specific types: age, ethnicity, gender, religion, other, and not cyberbullying.
- Compare the performance of ensemble methods (Hard Voting, Soft Voting, Stacking) against individual classifiers.
- Select the best model for real-time user input prediction.

3. Dataset

3.1 Overview

The dataset used is cyberbullying_tweets.csv, containing 47,692 labelled tweet records. Each record consists of two columns:

- tweet_text — the raw tweet content.
- cyberbullying_type — the target label indicating the nature of the tweet.

3.2 Class Distribution

The dataset contains six balanced classes with approximately 7,900–8,000 samples each:

Class Labels

age — Bullying related to a person's age
ethnicity — Bullying based on race or ethnicity
gender — Bullying targeting gender identity
religion — Bullying related to religious beliefs
other_cyberbullying — General cyberbullying not covered above
not_cyberbullying — Normal, non-harmful content

Total samples: 47,692 | Dataset dimensions: (47692, 2) | No missing values detected.

4. Methodology

4.1 Text Preprocessing

Raw tweet text contains noise such as hashtags, mentions, URLs, punctuation, and mixed casing. A custom clean_text() function was applied to normalize the text:

```
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'[^a-zA-Z]', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text
```

- Convert all text to lowercase for case-insensitive matching.
- Remove all non-alphabetic characters (numbers, punctuation, special symbols).
- Strip extra whitespace to produce clean, normalized tokens.

4.2 Feature Extraction — TF-IDF Vectorization

Term Frequency-Inverse Document Frequency (TF-IDF) was used to convert text into numerical feature vectors. The vectorizer was configured as follows:

- max_features = 5,000 — Top 5,000 most informative terms retained.
- ngram_range = (1, 2) — Unigrams and bigrams to capture context.
- stop_words = 'english' — Common English stop words removed.

4.3 Train-Test Split

The dataset was split into 80% training (38,153 samples) and 20% testing (9,539 samples) using stratified sampling (random_state=42) to preserve class proportions across splits.

4.4 Models Used

4.4.1 Logistic Regression

A linear classifier with L2 regularization trained with up to 1,000 iterations. class_weight='balanced' was set to handle any minor class imbalance. Logistic Regression is well-suited for high-dimensional sparse features like TF-IDF.

4.4.2 Linear SVC

A Support Vector Classifier with a linear kernel, optimized via the dual objective. Linear SVC is highly efficient for text classification tasks and scales well to large vocabulary sizes. class_weight='balanced' was also applied.

4.4.3 Hard Voting Ensemble

Combines Logistic Regression and Linear SVC by majority vote. Each classifier independently predicts the class label, and the final prediction is the class with the most votes.

4.4.4 Soft Voting Ensemble

Uses predicted class probabilities from each model and averages them before taking argmax. An SVC with probability=True (RBF kernel calibrated for probability output) was used alongside Logistic Regression. Soft voting generally outperforms hard voting when classifiers are well-calibrated.

4.4.5 Stacking Classifier

Logistic Regression and Linear SVC serve as base learners. Their predictions are fed as features to a meta-learner (Logistic Regression) that learns how to best combine them. Stacking can capture complementary strengths of base models.

5. Experimental Results

5.1 Model Performance Summary

All five models were evaluated on the 9,539-sample held-out test set. Results are as follows:

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	83.37%	83.84%	83.37%	83.51%
Linear SVC	82.65%	82.86%	82.65%	82.72%
Voting (Hard)	82.78%	82.88%	82.78%	82.81%
Voting (Soft)	**83.83%**	**84.14%**	**83.83%**	**83.93%**
Stacking	83.59%	83.99%	83.59%	83.67%

Best Model: Voting (Soft) — Accuracy: 83.83%, F1 Score: 83.93%

5.2 Detailed Classification Report — Stacking Classifier

The Stacking classifier (used as the final deployed model) achieved 83.59% accuracy. Per-class performance is detailed below:

Class	Precision	Recall	F1-Score	Support
age	0.97	0.98	0.97	1598
ethnicity	0.98	0.97	0.97	1592
gender	0.90	0.85	0.88	1595
not_cyberbullying	0.61	0.55	0.58	1589
other_cyberbullying	0.61	0.72	0.66	1565
religion	0.96	0.94	0.95	1600
accuracy			**0.84**	**9539**
macro avg	**0.84**	**0.84**	**0.84**	**9539**
weighted avg	**0.84**	**0.84**	**0.84**	**9539**

5.3 Key Observations

- Age, ethnicity, and religion classes achieved very high F1 scores (0.95–0.97), indicating that these categories have distinctive, learnable vocabulary.
- Gender-based cyberbullying achieved a moderate F1 of 0.88, suggesting more linguistic diversity in this category.
- not_cyberbullying and other_cyberbullying were the most challenging classes (F1: 0.58 and 0.66 respectively), likely due to subtlety and context-dependence.
- Soft Voting outperformed all other methods, benefiting from probability-weighted aggregation.
- Stacking performed comparably to Logistic Regression alone but at the cost of significantly higher training time and a convergence warning in the meta-learner.

6. Real-Time Prediction

The final deployed model is the Stacking Classifier. A predict_text() function was implemented to accept raw user input, preprocess it, vectorize it using the fitted TF-IDF transformer, and produce a predicted cyberbullying category:

```
def predict_text(text):
    text_cleaned = clean_text(text)
    text_vectorized = vectorizer.transform([text_cleaned])
    prediction = final_model.predict(text_vectorized)
    return prediction[0]
```

Sample Prediction

Input: "heyy"
Output: other_cyberbullying

Note: The model classified this ambiguous input as other_cyberbullying, suggesting that short or informal text with no clear context defaults to this catch-all category.

More representative testing with diverse inputs is recommended.

7. Discussion

7.1 Strengths

- Clean, balanced dataset of nearly 48,000 tweets ensures representative training.
- TF-IDF with bigrams captures basic context and phrase-level patterns.
- Ensemble methods provide robustness and minor accuracy improvements over individual models.
- Balanced class weights address class imbalance automatically.

7.2 Limitations

- Context-free features: TF-IDF does not capture semantic meaning or sarcasm, which are important in cyberbullying detection.
- Ambiguous classes: 'not_cyberbullying' and 'other_cyberbullying' overlap significantly, reducing model precision for these categories.
- Stacking meta-learner convergence warning: The default max_iter=100 is insufficient; this should be increased.
- No deep learning baseline: Transformer models like BERT or RoBERTa are likely to yield substantially higher performance on this task.

7.3 Future Work

- Apply transformer-based models (BERT, DistilBERT) for contextual text understanding.
- Add emoji, hashtag, and mention analysis as additional features.
- Expand the dataset and include multilingual tweets.
- Deploy the model as a REST API for integration with social media moderation pipelines.
- Explore SHAP or LIME for model interpretability.

8. Conclusion

This project successfully demonstrates that cyberbullying in tweets can be automatically detected and classified into six categories using TF-IDF features and ensemble machine learning models. The best-performing model — Soft Voting — achieved an accuracy of 83.83% and a weighted F1 score of 83.93%, which is competitive for a traditional ML pipeline on a six-class text classification problem.

While specific cyberbullying categories (age, ethnicity, religion) are classified with near-perfect accuracy, broader and more subjective categories remain challenging. The Stacking Classifier was chosen for deployment due to its interpretability and strong overall performance.

The work lays a solid foundation for more advanced NLP-based hate speech detection systems and highlights the effectiveness of ensemble methods in improving classification reliability.

9. Technology Stack

Component	Details
Language	Python 3.x
Data Handling	pandas, numpy
Visualization	matplotlib
NLP / Features	scikit-learn TF-IDF Vectorizer
Models	Logistic Regression, LinearSVC, SVC (probability)
Ensemble Methods	VotingClassifier, StackingClassifier
Evaluation	accuracy_score, classification_report, confusion_matrix
Environment	Anaconda / Jupyter Notebook

Git hub repo link:
https://github.com/saraswati-sanj/ML_FINAL_PROJECT/tree/main